

data-to-paper: Autonomous Research with a Verifiable Chain of Provenance

How a rule-based orchestrator drives interacting LLM agents through the full hypothesis-testing research path — data → hypothesis → analysis code → results → manuscript — while *programmatically chaining every number in the paper back to the line of code that produced it*. The contribution is not the autonomy; other systems have that. It is making the autonomous output **human-verifiable by construction**: a manuscript where you can click any reported odds-ratio and walk it back through a formula, a table cell, a code output file, to the exact lines of Python that generated it.

Background

Primary paper: Autonomous LLM-driven research from data to human-verifiable research papers (Tal Ifargan, Lukas Hafner, Maor Kern, Ori Alcalay, Roy Kishony — Technion, Apr 2024). Code: github.com/Technion-Kishony-lab/data-to-paper. Supplementary (runs, manuscripts, data-chained PDFs): github.com/rkishony/data-to-paper-supplementary.

data-to-paper is not a new model. Like Robin, it is an *orchestrator* — a deterministic, rule-based program (a "chained list of research steps") that wires stock ChatGPT models (gpt-3.5-turbo-0613, gpt-3.5-turbo-16k-0613, gpt-4) into a closed research loop. Its design framework is explicitly the **AI-in-science governance guidelines** of Bockting et al. (*Living guidelines for generative AI*, Nature 2023, ref 22), which demand accountability, oversight, and transparency. The whole system is an engineering answer to "how do you get those three properties out of a stochastic, hallucinating LLM."

Lineage it builds on:

1. **Multi-agent conversation frameworks** — AutoGen (Wu et al., 2308.08155) and MetaGPT (Hong et al., 2308.00352): the role-inverted performer/reviewer pattern.
2. **Self-Refine** (Madaan et al., 2303.17651) and **Check-Your-Facts** (Peng et al., 2302.12813): iterative LLM self-correction against feedback — here made rule-based, not LLM-judged.
3. **Semantic Scholar Academic Graph API** (Kinney et al., 2301.10140): external citation retrieval, so references are *retrieved*, not hallucinated. (Same anti-hallucination instinct as Robin's PaperQA2 retrieval, but lighter — an API call, not a RAG agent.)
4. **SPECTER** (Cohan et al., 2004.07180): title+abstract embeddings, used to measure that repeated runs cluster tightly.

Contemporary "AI scientist" systems it is positioned against: **The AI Scientist** (Lu et al., Sakana, 2408.06292) — end-to-end computational-ML papers, but no provenance layer; **AI Co-Scientist** (Gottweis et al., Google, 2502.18864) — hypothesis debate only, never touches data; **Robin** (Ghareeb et al., FutureHouse, 2505.13400) — wet-lab biology discovery loop. (data-to-paper predates the last two; the contrast below is retrospective.)

What Problem Does It Solve?

The other autonomous-research systems optimize for **discovery** — can the AI find something new? data-to-paper asks a different, prior question: **even if an AI produces a finished research paper, why should anyone trust it?**

This is the trust gap. An autonomous pipeline that emits a polished manuscript is *worse* than no tool if its numbers are subtly wrong, because the polish hides the error. The paper names the specific failure modes it must defend against: LLM hallucination injecting numbers that were never computed; *P*-hacking; and "overloading the publication system with medium-level and generic manuscripts." A system that just generates papers fast amplifies all three.

Contrast Robin concretely: Robin autonomously **wrote its own main-text data figures** — and that is presented as a feature, a sign of end-to-end capability. But there is no mechanism asserting that each plotted value traces back to the flow-cytometry code that produced it; you trust it because the lab validated the wet-lab result downstream. data-to-paper inverts the priority. It assumes the science will be **modest** (it openly says novelty "falls well behind high-end contemporary science") and spends its engineering budget instead on making every claim *mechanically auditable*. The thesis: autonomous research is only acceptable if it **enhances, rather than jeopardizes, traceability and verifiability** — so build the verification into the artifact itself.

Two design axes make this tractable:

- **fixed-goal vs open-goal** — is the research question human-provided (fixed) or invented by the system (open)? Open-goal is harder and where most errors live.
- **autopilot vs co-pilot** — does the human only approve each step (autopilot, overseeing), or actively inject review comments (co-pilot)? The headline result is about *when* you are forced to switch from the first to the second.

Key Terms

Research product: the typed output of a step — one of {Free text, LaTeX text, Structured text, Binary decision, Citations, Python code, Numerical data, PDF}. Everything flowing between steps is a typed product, which is what lets the orchestrator check it programmatically.

Performer conversation: one step = one fresh LLM conversation, pre-filled by the orchestrator with (i) a SYSTEM identity prompt ("You are a scientist who needs to write literature search queries"), (ii) a curated subset of prior products, (iii) a USER "mission prompt" stating the ask. The product is extracted from the reply.

Provided prior products: the *deliberately restricted* set of earlier products fed into a step. Not the whole history — only what that step needs. This is the central anti-hallucination lever: starving a step of irrelevant context prevents the LLM from mixing in unrelated numbers, and it is also what makes the information graph traceable.

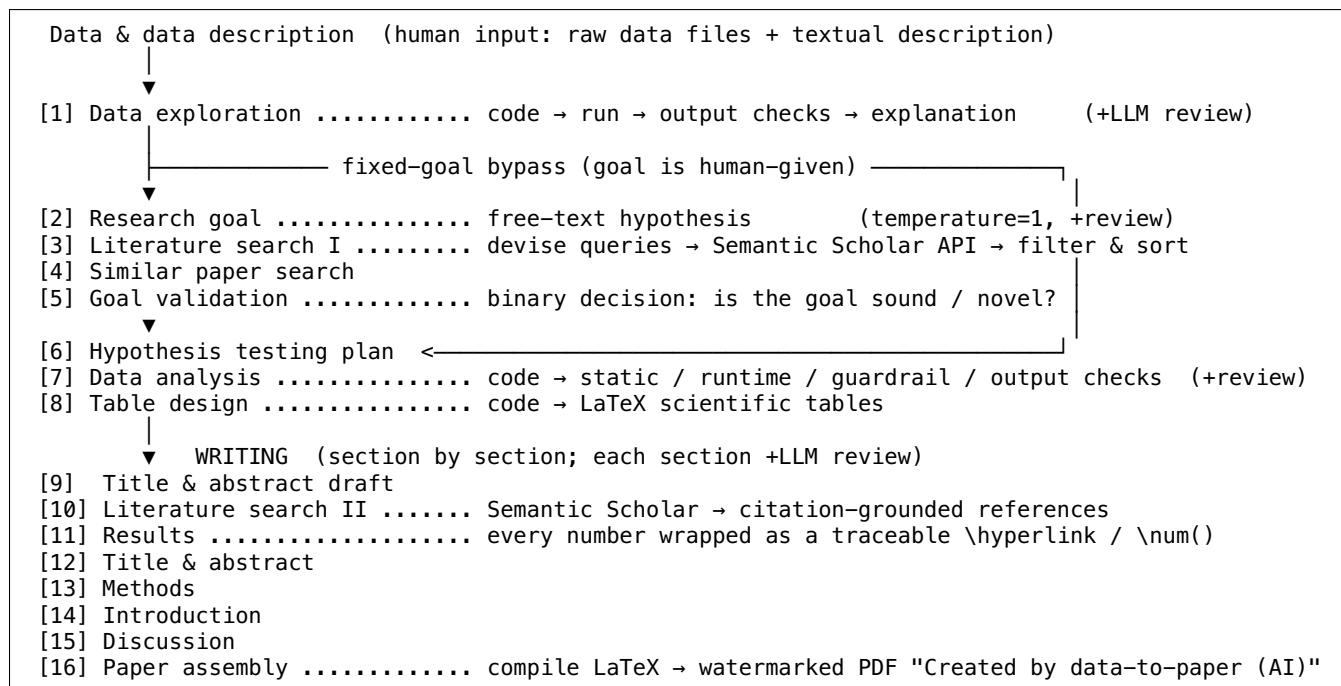
LLM-surrogating message: an ASSISTANT-attributed message written *programmatically by data-to-paper*, not by the model — e.g. fake acknowledgements ("Thank you for the Sounds.") inserted after each provided product so the conversation reads naturally. A small but load-bearing trick for controlling what the model "believes" it already said.

Rule-based review vs LLM review: two distinct gates. Rule-based = deterministic algorithmic checks (formatting, length, referencing, code static/runtime/output checks) producing programmatic feedback. LLM review = a second, role-inverted "reviewer" agent that critiques. A product must pass rule-based review; some steps also pass LLM review; in co-pilot mode a human review can be added on top.

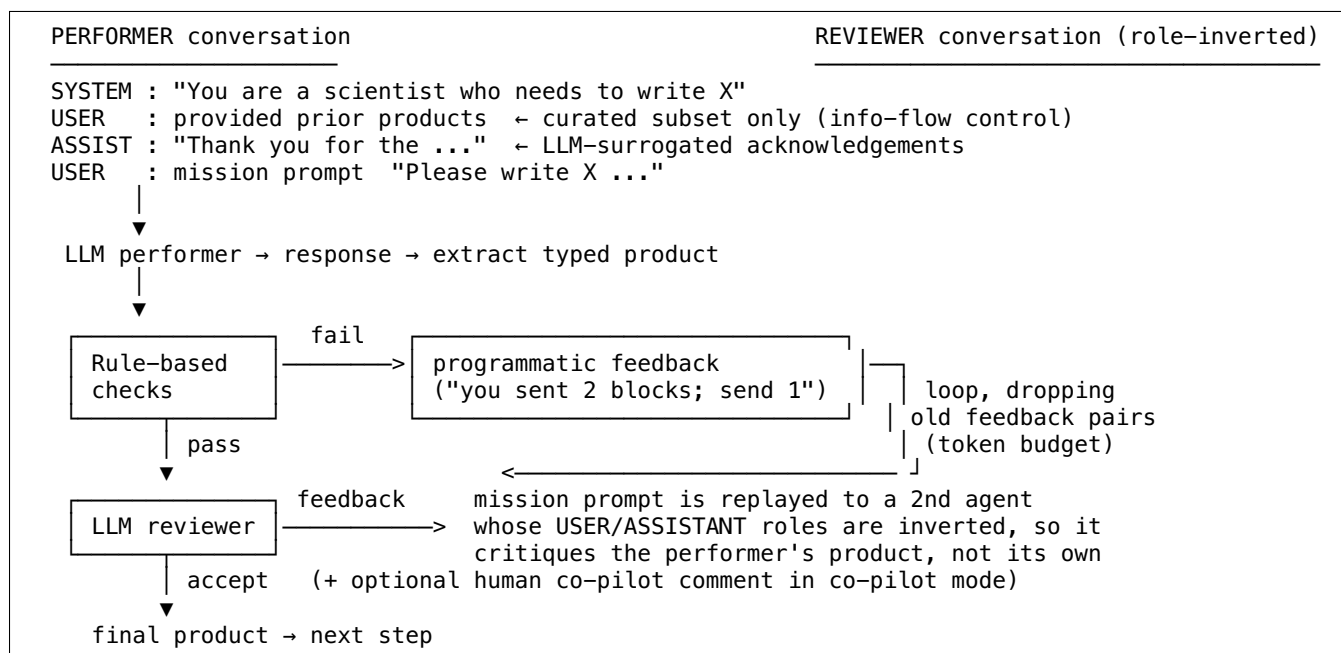
Data-chained manuscript: the output artifact — a PDF in which every cited numeric value is a recursively resolvable hyperlink back to its generating code. The verifiability deliverable.

The Stepwise Agent Workflow

~17 steps, grouped into three modules (Coding, Literature search, Writing), each step looping until its product passes review. The fixed-goal bypass skips goal-invention.



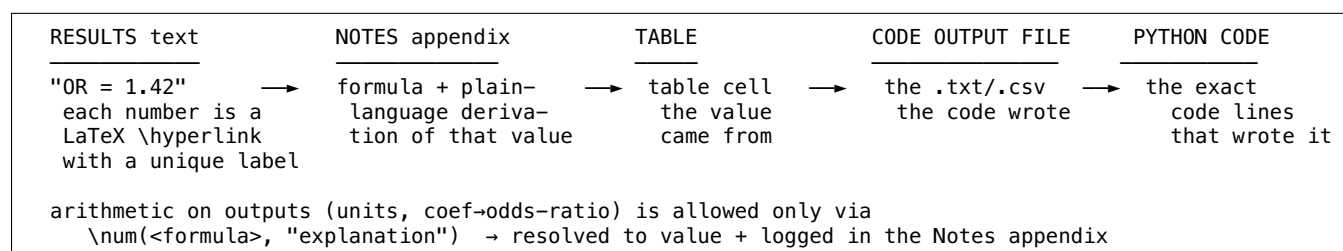
Each individual step runs the same internal control loop (Fig 1C / S1–S2):



Coding steps get the heaviest guardrails — four check levels: **static** (does the code match the requested structure?), **runtime** (syntax/runtime errors, warnings, banned operations caught during execution), **package-specific** (wrapped imports that block unsafe functionality and enforce e.g. *P*-value formatting), and **output** (are all requested output files present with the requested content?). Failures become feedback; the loop repeats. With open-source models this loop frequently *never converges* (see Results) — the guardrails are what make ChatGPT usable here at all.

The Provenance / Traceability Mechanism

This is the centerpiece and the thing no peer system has. The orchestrator tracks, end to end, which code lines produced which output, how outputs became table cells, and how table cells became numbers in the Results text. It then emits those links *into the PDF*.



How it is enforced (not merely hoped for):

- During the Results step, the orchestrator **assigns a unique label to every numeric value** present in the prior products and presents those numbers as LaTeX *hypertargets*. The mission prompt then *requires* the LLM to wrap each number it writes in a `\hyperlink` matching a label.
- A **rule-based check verifies** that every numeric value in the section is hyperlinked **and that each link's target label actually matches** the value in the provided-product context. A number with no valid source link fails review.
- Numbers that are derived (changing units, regression coefficient → odds ratio) must use `\num(<formula>, "explanation")`; on compilation these are replaced by the computed value and collected into the **Notes appendix** with their formula and explanation.
- **Hallucination tripwire**: if the model lacks a value it must emit the literal placeholder [unknown]. Detecting that placeholder (or any other) in the response **aborts the entire research cycle** rather than shipping a guessed number.

The net effect: a *data-chained manuscript* where vetting is mechanical. A human auditor clicks a number, lands on its formula, then its table, then the output file, then the generating code — no re-running, no reverse-engineering. The paper frames this as "setting a new standard for traceability for the coming era of AI-powered research." Where Robin earns trust *downstream* (validate the drug at the bench), data-to-paper earns it *in the artifact*.

Results & Success Rates

Four case studies; manuscripts manually vetted and color-highlighted (green=correct, yellow=imperfect, orange=minor error, red=major error).

Open-goal, autopilot — system invents the question. Two public datasets: "Health Indicators" (CDC BRFSS 2015, **253,680** responses × 22 diabetes features) and "Social Network" (117th US Congress Twitter interaction graph). 5 research cycles each, **~1 hour per cycle**, → 10 manuscripts. Result: **8/10 correct** (minor wording imperfections), **2/10 erroneous** with fundamental analysis/interpretation mistakes (one Health Indicators paper misinterpreted a correct analysis due to a hallucination in the goal-specification step; one Social Network paper ran an erroneous analysis → unfounded claims). All 10 followed canonical paper structure; all tables correctly represented the analysis; Health Indicators papers used logistic/linear regression with reasonable confounder choices.

Fixed-goal, autopilot — reproduce two peer-reviewed studies *published after the Sept-2021 ChatGPT knowledge cutoff* (so no memorization):

- **Challenge 1 — report both positive AND negative findings** (Saint-Fleur et al., NICU policy change, *PLoS One* 2023). 10 cycles: **all 10 correctly reproduced the analysis; 8/10 reached correct conclusions** reporting both positive and negative results; 2/10 had interpretation errors (1 affecting conclusions).
- **Challenge 2 — tunable-breadth multi-step ML** (Shim et al., predicting optimal pediatric tracheal-tube depth, 967 patients, *PLoS One* 2021). Here breadth = how many models to compare. The key finding: **error rate scales with task breadth**. On the *broad* original goal → **~90% error rate**. On *nearly identical* goals requesting **fewer models** → **10–20% error rate**. The number of Data-Analysis coding rounds rises with breadth (Fig 4), giving a built-in complexity alarm.

Human co-piloting closes the gap: on the hardest goal, **2–3 single-sentence review comments per run** (typically injected at the code-writing step) were enough to produce accurate papers **consistently** even for complex goals.

Headline: for simple research goals, fully-autonomous data-to-paper recapitulates peer-reviewed publications without major errors in **~80–90%** of runs; as goal complexity rises, the autonomous error rate climbs toward consistent failure, and **human co-piloting becomes critical** for reliability.

Why ChatGPT, not open-source (Fig S3, quantified): on the Research-goal step, **Llama 2-70B hallucinated features not in the dataset in all 10/10 runs**, while gpt-3.5-turbo used only real features. On the Data-analysis step (max 20 coding rounds): gpt-3.5/gpt-4 converged to working code in 1–3 rounds (median 1); **Llama 2-7B never converged**; CodeLlama converged once (after 8 rounds); Llama 2-70B converged in only 4/10 runs (13–18 rounds). Open-source models simply could not complete full cycles. Auto-escalation: the orchestrator upgrades a step to gpt-4 when rule-based feedback won't resolve, and to gpt-3.5-turbo-16k when the token budget is exceeded.

Run consistency (Fig 3): SPECTER title+abstract embeddings of all manuscripts form 4 tight, distinct clusters (one per case study); fixed-goal papers also cluster near their original published study — repeated stochastic runs converge on similar content.

Limitations (stated by the authors)

1. **Novelty is low.** The science "falls well behind high-end contemporary science." Papers are "not highly creative" — reasonable hypotheses tested with straightforward statistics. The achievement is *de novo* insight extraction from data with full provenance, not original mechanism.
2. **Hypothesis-testing only.** The system cannot formulate and pursue follow-up questions — no iterative result→new-hypothesis loop. (This is exactly Robin's signature capability and exactly what data-to-paper lacks. The two systems are near-complementary: Robin refines on results but has no provenance; data-to-paper has provenance but cannot refine.)
3. **Text and tables only.** No figures/plots in the output (again, contrast Robin's auto-generated figures), no non-tabular modalities.
4. **Not error-free autonomously.** ~10–20% fundamental-error rate even on *simple* tasks; consistent failure on complex tasks without a human.
5. **Reliability hinges on prompt/dataset framing.** Less detailed, less explicit research-goal and dataset descriptions raise analysis errors. The system inherits the brittleness of prompt engineering.
6. **Dual-use risk.** The authors flag that the same automation could enable *P*-hacking or flood journals with generic manuscripts — and position their verifiability features (data-chaining, AI watermark, unbiased positive/negative reporting, co-pilot oversight) as mitigations.

data-to-paper vs Alternatives

System	Domain	Produces full paper	Analyzes real data	Refines on results	Output verifiability (provenance)	Validated discovery
data-to-paper	tabular / stats	✓ (text + tables)	✓ (code, guardrailed)	✗ (no follow-up)	✓✓ number→code chain, enforced	✗ (recapitulation, not new)
Robin	wet-lab biology	~ (figures, not full paper)	✓ (Finch consensus)	✓ (in-context)	✗ (no claim→code chain)	✓ (ripasudil, in-vitro)
AI Scientist (v1/v2)	computational ML	✓	✓ (code only)	✓	✗ (none; known to hallucinate results)	✓ (a workshop paper)
AI Co-Scientist	biomedical	✗ (proposals only)	✗	✗ (no data)	✗	partial (others ran wet follow-ups)

The column that matters here is **provenance**. Every other system can be wrong in a way that is invisible in its output; data-to-paper's data-chaining makes a wrong number *findable*. That is the entire value proposition — it trades discovery ambition for auditability.

Practical Considerations / Reusable Ideas

Engineering patterns here transfer directly to any agentic-research or LLM-judge pipeline you build:

1. **Type your inter-step products and check them with rules, not vibes.** Deterministic rule-based review (formatting, referencing, code static/runtime/output) catches far more than an LLM critic and never hallucinates a pass. Reserve the LLM reviewer for judgment calls.
2. **Starve each step of context on purpose.** "Provided prior products" = only the curated subset a step needs. Less irrelevant context → fewer cross-contamination hallucinations, *and* a clean dependency graph you can trace.
3. **Make provenance a hard constraint, not documentation.** Assign every number a label, require the model to hyperlink it, then *programmatically verify the link target matches* — and abort on a [unknown] placeholder. Verifiability you enforce at extraction time is real; verifiability you ask for politely is not.
4. **Retrieve citations from an API; never let the model recall them.** Semantic Scholar call + a rule that bans memory-sourced citations = no fabricated references.
5. **Use task signals as a complexity alarm.** Coding rounds-to-convergence rose with task breadth — a cheap, automatic "this goal is too hard for autopilot, escalate to a human" trigger. Watch for analogous signals in your own loops.
6. **Know your switch point.** Autopilot is fine for narrow, well-specified goals (~80–90% clean); the moment breadth grows, 2–3 human sentences at the code step recover reliability. Design the co-pilot affordance in from the start.

When *not* to reach for this design: anything needing genuine novelty (it recapitulates, it does not invent), non-tabular data/figures, or research that requires following up on its own results (no hypothesis-refinement loop — use a Robin-style architecture there).

Key Papers

1. Ifargan, Hafner, Kern, Alcalay, Kishony. *Autonomous LLM-driven research from data to human-verifiable research papers*. arXiv 2404.17605 (2024). <https://arxiv.org/abs/2404.17605>
2. Bockting, van Dis, van Rooij, Zuidema, Bollen. *Living guidelines for generative AI — why scientists must oversee its use*. Nature 622:693–696 (2023). [design framework]
3. Wu et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv 2308.08155 (2023). <https://arxiv.org/abs/2308.08155>

4. Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. arXiv 2303.17651 (2023). <https://arxiv.org/abs/2303.17651>
5. Kinney et al. *The Semantic Scholar Open Data Platform*. arXiv 2301.10140 (2023). <https://arxiv.org/abs/2301.10140>
6. Cohan et al. *SPECTER: Document-level Representation Learning using Citation-informed Transformers*. arXiv 2004.07180 (2020). <https://arxiv.org/abs/2004.07180>
7. Lu et al. *The AI Scientist*. arXiv 2408.06292 (2024). <https://arxiv.org/abs/2408.06292>
8. Gottweis et al. *Towards an AI Co-Scientist*. arXiv 2502.18864 (2025). <https://arxiv.org/abs/2502.18864>
9. Ghareeb et al. *Robin: A multi-agent system for automating scientific discovery*. arXiv 2505.13400 (2025). <https://arxiv.org/abs/2505.13400> [companion guide: robin-guide.md]